

PROJECT FORESIGHT

Demand & Inventory Intelligence Complete Project Report

Client

NorthBay Living

Prepared by

Data Science Intern
Zidio Development — Internship Program

Report date

August 2026

Data period

14 August 2021 – 12 August 2026
200 SKUs · 365,000 daily records · ~2.01 million units sold

This report covers data cleaning, exploratory analysis, baseline and advanced forecasting, model evaluation, inventory risk scoring, business insights, and delivery artifacts.

Confidential — For internship evaluation only

Table of Contents

1. Introduction
 2. Business Context and Problem Statement
 3. Project Scope and Deliverables
 4. Data Overview and Schema
 5. Data Quality Assessment and Cleaning
 6. Exploratory Data Analysis
 7. Feature Engineering and Modelling Dataset
 8. Forecasting Methodology
 9. Baseline Model — Seasonal-Naive
 10. Advanced Model — LightGBM
 11. Evaluation and Model Selection
 12. Inventory Risk Scoring and Actions
 13. Business Insights
 14. Recommendations
 15. Dashboard and Deployment
 16. Limitations and Future Work
 17. Conclusion
- Appendix A — Metrics Summary
- Appendix B — Artifacts and Files
- Appendix C — Glossary

1. Introduction

Project FORESIGHT is a demand forecasting and inventory intelligence engagement for NorthBay Living. The project was completed as part of the Zidio Development data science internship. This document is the full project report. It explains the business problem, the data work, the models, the results, and the practical recommendations in clear professional language.

Retail teams often face two problems at the same time: running out of stock on items customers want, and holding too much stock on items that move slowly. Both problems cost money. Stockouts lose sales. Overstock locks capital and increases storage and markdown risk. A useful analytics system must therefore do two jobs: forecast demand in a measurable way, and turn that forecast into simple actions.

This project follows a baseline-first approach. A simple seasonal method is measured first. A stronger machine learning model is accepted only if it improves on that baseline. Forecasts are then combined with inventory positions to produce reorder and markdown recommendations.

Objectives of this project

1. Clean and document the client data extract so analysis is reliable and repeatable.
2. Describe demand, seasonality, promotions, and inventory patterns in plain language.
3. Build a weekly SKU-level forecast that beats a seasonal-naïve baseline on WAPE.
4. Score stockout and overstock risk and suggest actions for operations.
5. Estimate approximate rupee impact for prioritisation.
6. Deliver notebooks, reports, data files, and dashboard code for review and reuse.

2. Business Context and Problem Statement

NorthBay Living sells home and lifestyle products across categories such as Decor, Furniture, and Small Appliances. The catalogue in this dataset includes 200 active SKUs. History spans roughly five years of daily records. Demand is influenced by season, promotions, holidays, and product lifecycle (including launch dates).

When forecasts are weak or missing, replenishment becomes reactive. When inventory views are scattered, the team cannot quickly see which SKUs need orders and which SKUs are overweight. The business therefore needs a weekly demand signal at SKU level and a short, prioritised risk list.

Problem statement

How can NorthBay Living forecast weekly demand for each SKU more accurately than a simple baseline, and use that forecast with current stock to prioritise reorder and markdown actions with clear rupee context?

Success criteria used in this work

- Data quality issues are investigated, fixed or documented with reasons.
- EDA produces business insights, not only charts.
- A seasonal-naïve baseline is reported honestly.
- A stronger model beats the baseline on a time-based test set using WAPE.
- Risk scoring produces actionable SKU lists for operations.

3. Project Scope and Deliverables

The scope follows the FORESIGHT engagement structure. Work completed in this internship maps to the main delivery areas below.

Area	What was delivered	Status
Data pipeline	Cleaning rules, fixed calendar, cleaned daily file	Done

Area	What was delivered	Status
Quality & EDA	Quality report, EDA notebook, insight memo	Done
Forecasting	Weekly features, baseline, LightGBM, WAPE comparison	Done
Risk scoring	Action flags, ■ impact, risk CSV and report	Done
Packaging	Stage reports, executive report, dashboard script	Done

Not in this phase

- Live connection to ERP or warehouse systems
- Automatic purchase order creation
- Multi-location transfer optimisation

These can be added later on top of the weekly forecast and risk table.

4. Data Overview and Schema

The source extract was a single denormalised table. Sales facts, SKU attributes, inventory fields, and calendar fields appeared together. After cleaning, the working daily dataset has 365,000 rows and 200 SKUs from 14 August 2021 to 12 August 2026.

Grain and keys

The atomic grain is one row per SKU per day. Modelling uses a weekly grain: one row per SKU per week. Weekly aggregation reduces day-to-day noise and matches the planning frequency requested in the brief.

Important fields

Field	Role in the project
date, sku_id	Time and product keys
units_sold, revenue, unit_price	Demand and value measures
on_hand_units, on_order_units	Inventory position
lead_time_days, reorder_point	Replenishment context
promo_flag, promo_event, is_holiday	Demand drivers
category, subcategory, region	Product and location attributes
week, month, season, year	Calendar features rebuilt from date
is_stockout	True when on-hand units are zero

Headline volume: about 2.01 million units sold and about ■616.7 million total revenue across the history window. These totals are consistent between the cleaned daily file and the professional multi-sheet export built from it.

5. Data Quality Assessment and Cleaning

Before modelling, the extract was checked for missing values, inconsistent fields, duplicates, and logical issues. Each major issue received a clear action and a reason. Where a safe fix was not possible, the issue was left visible.

5.1 Junk columns

Trailing unnamed columns with no business meaning were removed so the schema stays close to the real data dictionary.

5.2 Missing unit price

Some rows had missing unit_price. When units were sold, price was calculated as revenue divided by units sold. Any remaining gaps were filled with list_price. After this step, unit_price had no missing values. This keeps revenue analysis and rupee impact calculations usable.

5.3 Missing on-order units

On-order quantity had more complex gaps. Values were filled only when the previous and next known values were the same. If neighbours disagreed, the cell was left null. 143 nulls remain. This avoids inventing inventory that cannot be justified.

5.4 Broken calendar fields

The original week, month, and season columns were unreliable. They matched the true date only a small share of the time. Using them for seasonality charts or features would have produced wrong conclusions. All three fields were rebuilt from the actual date: calendar month, ISO week number, and a simple season map (Winter: Dec–Feb, Spring: Mar–May, Summer: Jun–Aug, Fall: Sep–Nov). After the fix, month consistency is

100%.

5.5 Promotion labels

Missing promotion names were replaced with “No Promotion Recorded” when the promo flag was false, and “Unknown Promotion” when the promo flag was true. The fact that a promotion happened is preserved even when the event name is missing.

5.6 Pre-launch rows

Rows with date before the SKU launch date exist in the extract. They show zero sales. They were kept in the cleaned file for completeness and flagged in documentation. They do not create false demand; they can be filtered out for modelling if a strict post-launch window is preferred.

5.7 Duplicates, signs, and revenue checks

No duplicate date+SKU keys were found. No negative values were found in core numeric measures. Small differences between revenue and units times unit price were consistent with rounding and were left unchanged.

5.8 Cleaned data summary

Check	Result
Rows	365,000
SKUs	200
Remaining nulls	143 in on_order_units only
Calendar month match to date	100%
Duplicates	0

6. Exploratory Data Analysis

Exploratory analysis was used to understand how demand and inventory behave before any model was trained. The aim was to support both technical design choices and business conversation.

6.1 Sales distribution

Units sold per day are right-skewed. Many days have low or moderate volume; fewer days and fewer SKUs produce very high volume. Revenue is more skewed still. In practical terms, a small part of the assortment and calendar drives a large share of value. Forecasting and stock attention should not treat every SKU as equal.

6.2 Trend and seasonality

Across five years, activity shows overall growth and repeated seasonal movement. After correcting calendar fields, monthly and seasonal patterns are usable. This is why the modelling design includes lag_52 (same week last year) and calendar features such as month and week of year.

6.3 Category performance

Furniture and Small Appliances are major contributors to revenue in this dataset. Decor is also material. Subcategory views show that a limited set of product types dominate. Category differences also appear later in model error rates.

6.4 Concentration (80/20 style pattern)

Revenue is concentrated in a minority of SKUs. This is common in retail. It means that improving forecast accuracy and availability on the top SKUs protects more sales than spreading effort evenly across all 200 items.

6.5 Promotions and holidays

Average revenue is higher on promotion days than on non-promotion days. Named campaign periods stand out. Holiday flags also lift demand. These signals are kept as features and should remain visible in planning calendars.

6.6 Inventory and stockouts

Over the full history, 198 of 200 SKUs reached zero on-hand stock at least once. Stockouts have been a structural issue, not a rare event. In a recent 90-day window, pure dead stock under strict rules (zero recent sales with stock still held) was not the main pattern. The stronger near-term message is availability risk on active items, not a large set of completely inactive stock.

6.7 What EDA changed in the modelling plan

- Prefer weekly grain with seasonal lag.
- Include promo and holiday features.
- Expect harder forecast error on some categories.
- Connect forecast to inventory so outputs are actionable.

7. Feature Engineering and Modelling Dataset

Daily data was aggregated to SKU × week. Weeks were defined ending on Sunday. For each SKU-week the pipeline stored total units, total revenue, average price, end-of-week on-hand and on-order, lead time, reorder point, promo day count, holiday day count, and category attributes. The weekly table has about 52,400 rows.

Feature groups

Group	Features	Why included
Lags	lag_1, lag_2, lag_4, lag_52	Recent demand and last-year seasonality

Group	Features	Why included
Rolling	roll_mean_4, roll_mean_8, roll_std_4	Smooth level and volatility
Calendar	weekofyear, month, quarter	Seasonal structure
Drivers	promo_days, holiday_days	Known lifts
Context	price, on_hand, lead_time, reorder_point, category	Commercial and stock context

Leakage control: lag and rolling features are computed with a shift so that only past weeks are used. This is essential for a fair test and for real use, where future sales are unknown.

8. Forecasting Methodology

The methodology follows three rules from the project brief: start with a baseline, measure with WAPE, and validate with a time-based design that does not shuffle the future into the past.

8.1 Why a baseline first

A complex model can look impressive and still fail to beat a simple seasonal rule. Reporting the baseline first keeps the project honest and gives a clear target for improvement.

8.2 Metric — WAPE

$WAPE = (\text{sum of absolute forecast errors}) / (\text{sum of absolute actuals}) \times 100$. Lower values are better. WAPE puts more weight on high-volume SKUs and weeks, which matches commercial impact better than treating every SKU-week equally.

8.3 Train and test design

The last 52 weeks form the test set. All earlier eligible weeks form the training set. This mirrors production use: train on history, score the next year-like window, then compare models on the same window.

9. Baseline Model — Seasonal-Naive

Seasonal-naive prediction for a SKU-week is the units sold in the same week of the previous year (lag_52). If lag_52 is missing, the method uses lag_4, then lag_1, then zero. The rule is simple to explain to business users.

Baseline performance

Slice	WAPE
Overall evaluation window	32.12%
Last 1 year	31.26%
Decor	30.02%
Furniture	31.72%
Small Appliances	36.84%

Interpretation: copying last year's week gets error into the low thirties in percentage WAPE terms. Small Appliances is the weakest category for this simple rule. This baseline is the official bar for the advanced model.

10. Advanced Model — LightGBM

LightGBM is a gradient boosting algorithm. It builds many small decision trees one after another. Each tree focuses on fixing errors left by the previous trees. It is widely used for tabular forecasting problems because it handles mixed feature types well and trains efficiently.

How it was used here

The model received the engineered weekly features listed earlier. Early rows without basic lags were removed before training. Training used history before the final 52 weeks. The test set was the final 52 weeks only. Predictions below zero were set to zero because demand cannot be negative. Early stopping reduced overfit risk.

Why LightGBM was a fit for this project

- Works well with lag, calendar, and promo features
- Fast enough for internship iteration cycles

- Provides feature importance for explanation
- Easy to compare against the baseline on the same test window

11. Evaluation and Model Selection

Headline comparison (last 52 weeks test set)

Model	WAPE
Seasonal-Naive baseline	31.26%
LightGBM	19.96%
Improvement	11.29 points lower error

Category detail on the same test set

Category	LightGBM WAPE	Baseline WAPE
Decor	18.77%	29.10%
Furniture	19.01%	28.97%
Small Appliances	24.41%	40.58%

LightGBM improves every category. The largest absolute improvement appears in Small Appliances, which was also the weakest baseline category. Decor and Furniture land near 19% WAPE on the test window.

Feature importance (business reading)

The most important features were lag_1 (last week), lag_52 (same week last year), on_hand, reorder_point, additional short lags, rolling means, and price. This is intuitive: recent sales, seasonal memory, and stock context drive the forecast.

Selection decision

Because LightGBM beats the seasonal-naive baseline by a wide margin on the agreed metric and test design, it is selected as the forecast used for inventory risk scoring and planning outputs.

12. Inventory Risk Scoring and Actions

Forecast accuracy alone does not move inventory. The risk module combines the selected forecast with current stock and lead time to classify each SKU into an action bucket.

Calculations

- Inventory position = on-hand units + on-order units
- Lead time in weeks = lead time days divided by 7 (minimum one week)
- Demand during lead time = weekly forecast × lead time in weeks
- Weeks of cover = inventory position ÷ weekly forecast (when forecast is positive)

Decision rules used

- If inventory position is below demand during lead time → REORDER (stockout risk)
- If weeks of cover are above 12 → MARKDOWN / REDUCE (overstock risk)
- If neither applies → HEALTHY
- If both flags appear → WATCH for manual review

Sales at risk is approximated as forecast × average price for REORDER SKUs. Locked capital is approximated from excess units above an eight-week cover idea using a simple cost proxy. These are planning estimates for prioritisation, not formal accounting figures.

Latest week results

Action	SKU count	Signal
HEALTHY	193	No urgent action
REORDER	5	Sales at risk ≈ ■99,745
MARKDOWN / REDUCE	2	Locked capital ≈ ■197,699

Examples: SKU0144 and SKU0143 appear at the top of the reorder list by sales at risk. SKU0190 dominates the markdown list by locked capital. Because only seven SKUs are non-healthy in this snapshot, the operations list is short enough to act on quickly.

13. Business Insights

1. The advanced weekly model is not only more accurate than the baseline; the gap is large enough to matter for planning quality.
2. Stockouts were historically widespread, but the current risk snapshot is concentrated. Focused action is possible.
3. Revenue concentration means the business gains more by protecting availability on top SKUs than by treating all SKUs equally.
4. Promotions and holidays are real demand drivers and should remain in both the model and the commercial calendar.
5. Raw calendar fields in extracts can be wrong. Validating and rebuilding them was necessary before seasonal analysis and modelling.
6. A forecast becomes valuable when it is paired with inventory and turned into REORDER / MARKDOWN / HEALTHY language.

14. Recommendations

Immediate operations

- Review the five REORDER SKUs and place or expedite orders where lead times are long.
- Review the two MARKDOWN SKUs, especially the furniture SKU with high locked capital, for commercial actions.

Process

- Refresh the weekly forecast on a fixed cadence (for example every Monday).
- Keep a standing risk scorecard with the same action names so the team builds habit.
- Track WAPE over time so model quality does not silently degrade.

Analytics roadmap

- Consider category-specific models if Small Appliances remains harder after more tuning.
- Tune weeks-of-cover thresholds with operations and finance stakeholders.
- Optionally deploy the Streamlit app so non-technical users can filter the action list without notebooks.

15. Dashboard and Deployment

A Streamlit application script (app.py) presents KPIs, action mix, rupee impact, a filterable risk table, top reorder and markdown charts, and a single-SKU actual versus forecast view. It reads the risk CSV and the weekly prediction file.

Local run: place app.py with the two CSV files in one folder, install streamlit and plotly, then run streamlit run app.py from that folder. For a shared URL, the same app can be deployed on Streamlit Community Cloud from GitHub. Confirm with the mentor whether a live URL is mandatory or whether a local demo and screenshots are enough for evaluation.

16. Limitations and Future Work

- 143 on_order nulls remain and must be handled carefully in any automation.
- Risk thresholds are starting points and should be validated with the business.
- Rupee impact values are approximate prioritisation aids, not audited financial results.
- One global model was used; specialised models per category may add further lift.
- This phase is batch-oriented; live system integration is future work.

Useful extensions include richer rolling-origin backtests, hierarchical reconciliation, automated weekly jobs, and a light API over the risk table.

17. Conclusion

This project took NorthBay Living from a raw, imperfect extract to a cleaned data foundation, a documented baseline, a stronger weekly forecast, and an inventory risk layer with prioritised actions. On the main test window, LightGBM reduced WAPE from 31.26% to 19.96%, beating the seasonal-naive baseline by 11.29 points. The latest risk snapshot highlights five reorder SKUs and two markdown SKUs, with approximate sales at risk near ■1.0 lakh and locked capital near ■2.0 lakh.

The internship deliverables include cleaned data, modelling tables, notebooks, stage reports, an executive summary, this full report, and dashboard code. Together they meet the core FORESIGHT aims: measurable forecast improvement, transparent evaluation, and practical inventory decision support.

Appendix A — Metrics Summary

Metric	Value
Daily rows after cleaning	365,000
Number of SKUs	200
History window	2021-08-14 to 2026-08-12
Weekly modelling rows	52,400
Baseline WAPE (last year)	31.26%
LightGBM WAPE (last year)	19.96%
WAPE improvement	11.29 points
HEALTHY SKUs (latest week)	193
REORDER SKUs (latest week)	5
MARKDOWN SKUs (latest week)	2
Approx. sales at risk	■99,745
Approx. locked capital	■197,699

Appendix B — Artifacts and Files

Data files

- retail_forecasting_cleaned_v2 — cleaned daily dataset
- Retail_Forecasting_Cleaned_Professional.xlsx — professional multi-sheet workbook
- weekly_forecast_ready.csv — weekly features and baseline
- weekly_with_lgb_predictions.csv — weekly data with LightGBM predictions
- sku_risk_scores.csv — latest risk actions and impact fields

Notebooks

- Data cleaning notebook
- EDA notebook
- Feature engineering and baseline notebook
- LightGBM model notebook
- Risk scoring notebook

Reports

- Data Quality Report
- EDA Insight Memo
- Feature and Baseline Report
- LightGBM Model Report
- Risk Scoring Report
- Executive Final Report
- This Full Project Report

Application

- app.py — Streamlit dashboard

Appendix C — Glossary

Term	Meaning in this project
SKU	Stock keeping unit — one product identity
WAPE	Weighted Absolute Percentage Error — main accuracy metric
Seasonal-naive	Forecast = same week last year
LightGBM	Gradient boosting model used as the advanced forecast
Lead time	Days needed for replenishment after order
Weeks of cover	How many weeks current stock can support at forecast demand
REORDER	Action flag for stockout risk
MARKDOWN / REDUCE	Action flag for overstock risk
Sales at risk	Approximate value exposed when stock may run out
Locked capital	Approximate value tied in excess stock

End of Report — Project FORESIGHT | NorthBay Living | Zidio Development Internship | August 2026
Confidential — For internship evaluation only

Appendix D — Detailed Implementation Notes

This appendix records practical implementation choices so another analyst can reproduce the pipeline.

D.1 Cleaning sequence

1. Load the raw extract and drop empty unnamed columns.
2. Parse date and launch_date as datetime values.
3. Impute unit_price from revenue/units_sold, then list_price.
4. Apply cautious fill logic for on_order_units; leave ambiguous rows null.
5. Rebuild week, month, and season from date.
6. Standardise promo_event labels using promo_flag.
7. Create is_stockout where on_hand_units equals zero.
8. Save cleaned daily output and document remaining limitations.

D.2 Weekly aggregation rules

Units and revenue are summed within each SKU-week. On-hand and on-order use the last value in the week. Lead time and reorder point use a mean or stable SKU-level value. Promo and holiday contributions are counted as days. Category attributes are taken as the first value in the group. Week start is stored as a timestamp for sorting and plotting.

D.3 Model training notes

Rows missing lag_1 or lag_4 were dropped before training so the model always sees core lag signals. Category was converted to a simple integer code. Early stopping monitored the test set mean absolute error style metric inside LightGBM's training loop. Final reported quality uses WAPE on the held-out 52 weeks, which is the business metric of record.

D.4 Risk scoring notes

Where LightGBM predictions exist for the latest week they are preferred; otherwise the baseline value is used as a fallback. Forecast values are clipped at zero. Lead time days are converted to weeks with a floor of one week so very short lead times still create a minimum coverage demand. Thresholds for overstock can be changed later without retraining the demand model.

Appendix E — Alignment with FORESIGHT Requirements

The table below maps project requirements commonly stated in the FORESIGHT brief to evidence in this internship delivery.

Requirement theme	Evidence in this project
Clean imperfect extracts	Quality section + cleaned_v2 + professional workbook
Document data decisions	Data quality report and Section 5 of this report
EDA with business insights	EDA notebook, insight memo, Section 6 and 13
Baseline-first forecasting	Seasonal-naive model and reported WAPE
Beat baseline on WAPE	LightGBM 19.96% vs baseline 31.26%
Weekly SKU-level forecast	52,400 weekly rows and prediction files
Risk decision grid	REORDER / MARKDOWN / HEALTHY outputs
Rupee impact view	Sales at risk and locked capital estimates
Usable for non-technical readers	Executive report, full report, dashboard script

This mapping is intended to help evaluators verify completeness against the original engagement expectations.

Appendix F — How to Read the Outputs

For a business reviewer

Start with the Executive Final Report or Sections 11–14 of this document. Focus on the WAPE improvement, the latest action counts, and the recommended next steps. Use `sku_risk_scores.csv` as the working action list.

For a technical reviewer

Start with Sections 5, 7, 8, and 11. Check leakage controls, train/test split, and metric definition. Reproduce key numbers from the notebooks and CSV outputs listed in Appendix B.

For an operations user

Filter `sku_risk_scores.csv` to REORDER and MARKDOWN rows. Sort by `sales_at_risk` or `locked_capital`. Treat the values as prioritisation help, then confirm with live stock and supplier constraints before ordering or discounting.

Suggested weekly operating rhythm

1. Update cleaned weekly features with the newest closed week.
2. Refresh LightGBM predictions for the planning horizon used by the team.
3. Rebuild the risk table with current on-hand and on-order values.
4. Share the REORDER and MARKDOWN lists in the operations meeting.
5. Log actual actions taken so the process can be improved next cycle.

Final note: All major numerical results in this report (row counts, WAPE values, action counts, and rupee estimates) are taken from the executed internship notebooks and verified outputs available in the project package.

Appendix G — Declaration and Submission Notes

This report and the supporting notebooks represent the internship work completed for Project FORESIGHT under the Zidio Development program for the NorthBay Living case context. Numerical results are taken from executed Python notebooks using the cleaned project data.

What to submit together with this report

- This full project report (PDF)
- Stage reports already prepared (quality, EDA, baseline, LightGBM, risk, executive summary)
- Notebooks for cleaning, EDA, features/baseline, LightGBM, and risk scoring
- Key CSV outputs: cleaned daily data, weekly modelling table, predictions, risk scores
- Dashboard script app.py and requirements list
- Optional: GitHub repository link if the mentor requested version control evidence

Reading order for evaluators

1) Cover and Sections 1–3 for context. 2) Section 11 for the main model result. 3) Section 12 for operational actions. 4) Appendices for metrics and file lists. 5) Notebooks only if deep technical verification is required.

Language note: This document is written in clear professional English so both technical and business reviewers can follow the story without unnecessary jargon. Where technical terms are required, they are defined in the glossary.

Thank you for reviewing this Project FORESIGHT delivery. The package is ready for internship evaluation.